Table of Contents

Introduction **2**

Frontend Item Creation and Http Post **3**

ItemListPage.tsx *3*

ItemForm.tsx *4*

types/item.tsx *5*

ItemCreatePage.tsx *7*

App.tsx *8*

ItemAPIController create method **9**

Inspect the error logging and input validation **10**

Demo: Http Post

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT/Gemini/Claude.

Introduction

This demo will guide you through the process of creating an item from the client side using an HTTP POST request to a backend API.

Frontend Item Creation and HTTP Post:

1. **Modularise the Form:** Create two files under `src/items` for modularising the `ItemForm` and `ItemCreatePage` logic, as the same form will be used for item updates.
2. **ItemForm Component**:
    - **ItemFormProps:** Input properties including an event handler `onItemChanged` for item creation or updating, plus optional `itemId`, `isUpdate`, and `initialData` (so the same form works for both create and update).
    - **State Management:** Defines local `useState` for each item field (name, price, description, imageUrl). This is UI state local to the form, not server state, so it stays as plain `useState` even though data fetching elsewhere uses React Query.
    - **Event handling `onCancel`:** Implements `onCancel` to navigate to the previous page, and `handleSubmit` to manage form submission.
    - **Event handling submit:** Collects form data into an `item` object and triggers the `onItemChanged` function passed from parent components — the parent decides whether that means create or update.
    - **Form Structure:** Includes form groups for item properties with input validation mechanisms and buttons for submitting and cancelling.
3. **ItemCreatePage: Handle Item Creation:** Use a `useMutation` hook (from `@tanstack/react-query`) with `ItemService.createItem` as its `mutationFn`. On success, invalidate the `['items']` query so the item list refetches automatically, then navigate back to `/items`. On error, handle it in `onError`. Pass `handleItemCreated = (item) => createMutation.mutate(item)` down to `ItemForm` as `onItemChanged`.
4. **Update Routing**: Add a route entry in App.tsx for ItemCreatePage.

Backend API Controller:

1. **HTTP POST Endpoint**: Handles POST requests to create a new item as ItemDto.
2. **Validation**: Checks if the item data is null.
3. **Item Conversion**: Converts the ItemDto from the request body to an Item entity.
4. **Database Operation**: Uses repository method to save the item.

Testing and Validation:

1. Use browser console to inspect item creation
2. Test form validation messages.
3. Use Scalar UI to test the backend API for item creation.

Frontend Item Creation and Http Post

First add a Button in `ItemListPage.tsx`. Rather than react-bootstrap's `Button`, we use react-router's `Link` component directly, styled with Bootstrap's own CSS classes (`btn`, `btn-secondary`).

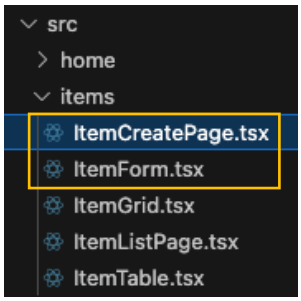
ItemListPage.tsx

ItemListPage.tsx U

6-React-Intro > Demo-react-6-httppost > myshop > src > items > ItemListPage

```tsx
import { useState } from 'react';
import { useQuery } from '@tanstack/react-query';
import { Button, Form } from 'react-bootstrap';
import ItemTable from './ItemTable';
import ItemGrid from './ItemGrid';
import { Item } from '../types/item';
import { Link } from 'react-router-dom';
return (
  <div className="text-start">
    <h1>Items</h1>
    <Button onClick={() => refetch()} variant="primary" className="mb-3 me-2" disabled={isLoading}>
      {isLoading ? 'Loading...' : 'Refresh Items'}
    </Button>
    <Button onClick={toggleTableOrGrid} variant="primary" className="mb-3 me-2">
      {showTable ? `Display Grid` : 'Display Table'}
    </Button>
    <Form.Group className="mb-3">
    <Form.Control
      type="text"
      placeholder="Search by name or description"
      value={searchQuery}
      onChange={e => setSearchQuery(e.target.value)}
    />
    </Form.Group>
    {error && <p style={{ color: 'red' }}>Failed to fetch items.</p>}
    {showTable
    ? <ItemTable items={filteredItems} apiUrl={API_URL} />
    : <ItemGrid items={filteredItems} apiUrl={API_URL} />}
    <Link to="/itemcreate" className="btn btn-secondary mt-3">Add New Item</Link> {/* this is a button */}
  </div>
);
```

Create two files under `src/items`. We modularise the form for inputting an item and the page logic to handle it, right from the beginning, because `ItemUpdate` will use the same form.



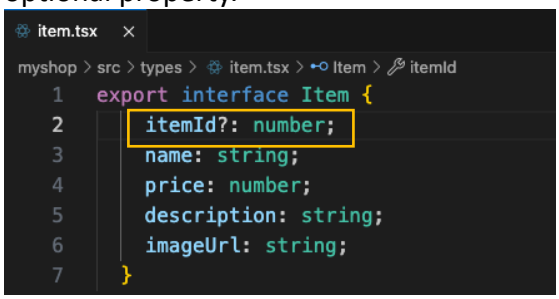
ItemForm.tsx



The interface `ItemFormProps` is similar to a class definition in C# in that it explicitly defines the properties expected by the React function component. In this interface, the `itemId` property has a question mark (`?`), which indicates that it is optional. This is appropriate because when creating a new item from the frontend, the `itemId` is not yet known; the backend (typically the database) automatically assigns it upon creation. `ItemFormProps` also includes optional `error` and `submitting` properties, which let the parent page (`ItemCreatePage` or `ItemUpdatePage`) pass down the current status of its network request, so the form can show a loading state on the submit button and display any error message, without needing to manage that state itself.

types/item.tsx

Accordingly, we need to modify `myshop/src/types/item.tsx` to accommodate such an optional property.



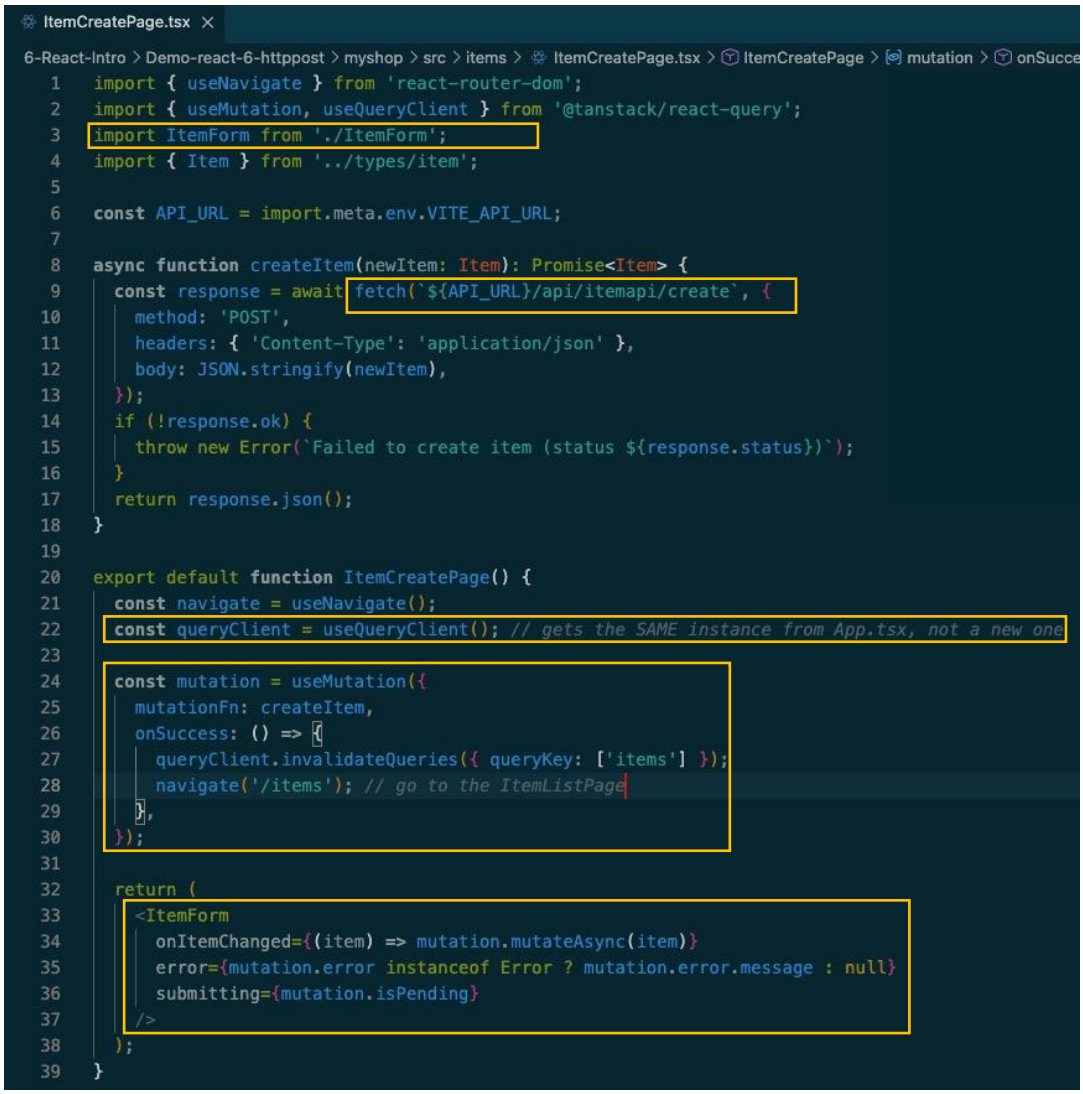
The `ItemForm` component defines an interface of its properties, including an event `onItemChanged` (which should trigger creating or updating an item), an optional `itemId` (empty in case of item creation, non-empty in case of item updating), and the `error`/`submitting` status passed down from the parent.

- `ItemForm` takes its props, including `onItemChanged`, `itemId`, `initialValues`, `error`, and `submitting`.
- It defines a set of typed states for the four item properties. Note that `ItemForm` does not keeps its own `error` state, the error to display is now supplied by the parent as a prop, since the parent owns the network request and therefore knows whether it succeeded or failed.
- It defines a function `onCancel` that handles the event when the Cancel button is clicked. It navigates to the previous page in the browser history.
- It defines a function that handles the submit event.
    - In web forms, the default action when a user submits a form is to reload the page. By calling `event.preventDefault()` inside a form submission handler, you can prevent this page reload. Note the event type here is `React.SyntheticEvent<HTMLFormElement>`.
    - It creates an item using the form data.
    - It calls (and awaits) the `onItemChanged` function with the input argument item. The `onItemChanged` function is passed from another component (`ItemCreatePage` or `ItemUpdatePage`) as an input argument, and should have a different effect depending on which page passes it in. Because `handleSubmit` awaits this call, any rejection from `onItemChanged` simply propagates up to be handled by whichever mutation logic the parent page has set up. Thus, `ItemForm` itself no longer needs a `try/catch`.
    - It returns a JSX component, elaborated next.

The form has four **Form Groups**, a **Button** for submit, and a Button for cancel. In each **Form Group**, there is a label, input area, placeholder text, value, and the event that is triggered when the value changes. On the `item.name` and `item.price` **Form Groups**, there are input validation mechanisms: `required`, `pattern`, `min`, `step`, and an error message title. The submit **Button**'s label and disabled state now reflect the `submitting` prop, showing "**Saving**..." while a request is in flight and preventing duplicate submissions; below the form fields, the `error` prop (if present) is displayed in an `Alert`.

```
43    return (
44      <Form onSubmit={handleSubmit}>
45        <Form.Group controlId="formItemName" className="mb-3">
46          <Form.Label>Name</Form.Label>
47          <Form.Control
48            type="text"
49            placeholder="Enter item name"
50            value={name}
51            onChange={(e) => setName(e.target.value)}      input validation
52            required
53            pattern="[0-9a-zA-ZæøåÆØÅ. \-]{2,20}"
54            title="The Name must be numbers or letters and between 2 to 20 characters.
              "
55          />
56        </Form.Group>
57
58        <Form.Group controlId="formItemPrice" className="mb-3">
59          <Form.Label>Price</Form.Label>
60          <Form.Control
61            type="number"
62            placeholder="Enter item price"
63            value={price}
64            onChange={(e) => setPrice(Number(e.target.value))}      input validation
65            required
66            min="0.01"
67            step="0.01"
68          />
69        </Form.Group>
70
71        <Form.Group controlId="formItemDescription" className="mb-3">
72          <Form.Label>Description</Form.Label>
73          <Form.Control
74            as="textarea"
75            rows={3}
76            placeholder="Enter item description"
77            value={description}
78            onChange={(e) => setDescription(e.target.value)}
79          />
80        </Form.Group>
82        <Form.Group controlId="formItemImageUrl" className="mb-3">
83          <Form.Label>Image URL</Form.Label>
84          <Form.Control
85            type="text"
86            placeholder="Enter image URL"
87            value={imageUrl}
88            onChange={(e) => setImageUrl(e.target.value)}
89          />
90        </Form.Group>
91
92        {error && <Alert variant="danger">{error}</Alert>}
93
94        <Button variant="primary" type="submit" disabled={submitting}>
95          {submitting ? 'Saving...' : 'Create Item'}
96        </Button>
97        <Button variant="secondary" onClick={onCancel} className="ms-2" disabled=
          {submitting}>
98          Cancel
99        </Button>
100     </Form>
101   );
102 }
```

ItemCreatePage.tsx



Rather than manually managing loading and error state with `useState` and a raw `fetch` call inside a `try/catch`, `ItemCreatePage` uses React Query's `useMutation` hook, since the app already uses React Query (via `QueryClientProvider` in `App.tsx`) for managing server state.

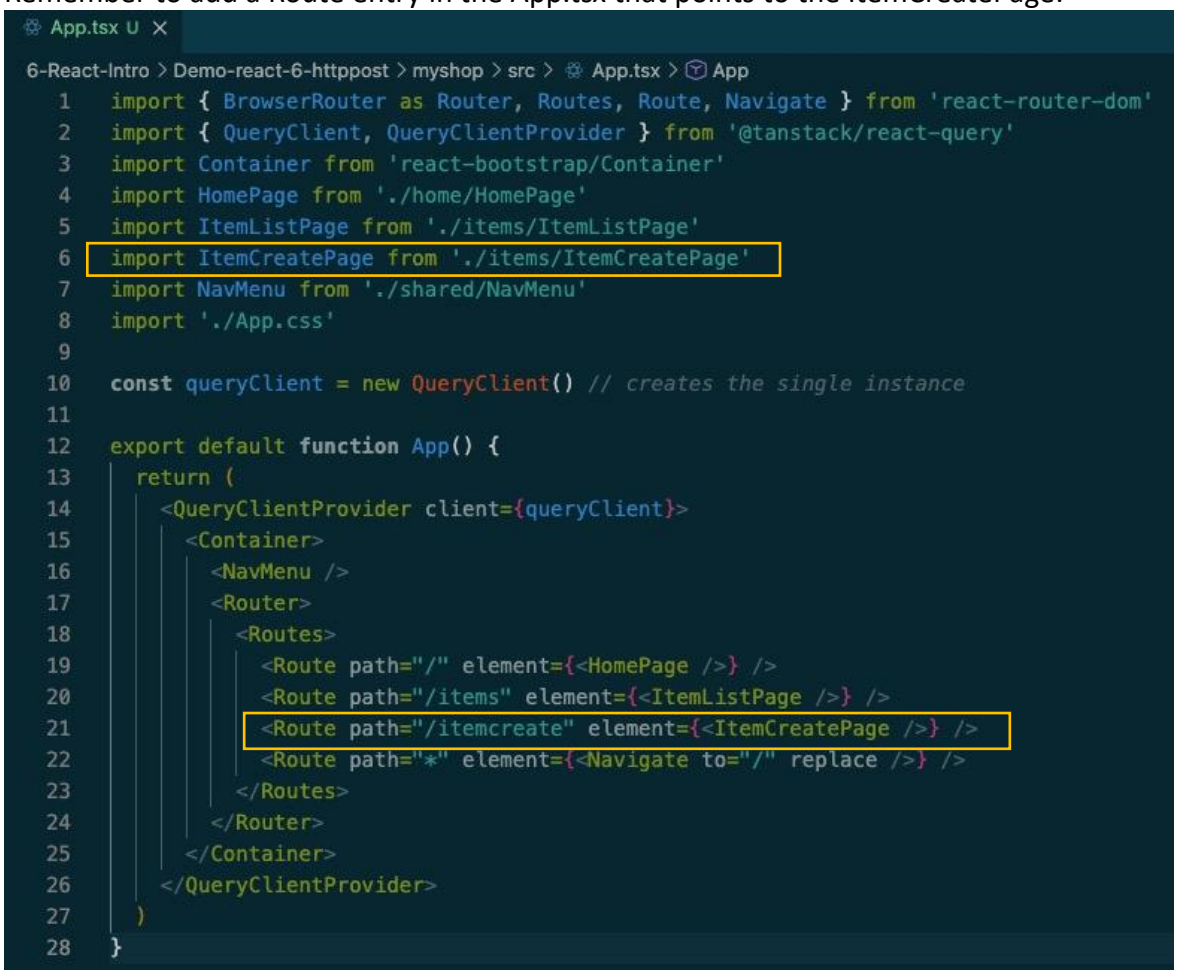
In server-state terminology, **queries** read data, while **mutations** create, update, or delete data on the server (because they change/mutate the data). The `useMutation` hook is designed specifically for these side-effect operations. It tracks the request lifecycle automatically, giving us built-in `isPending` and `error` states, and allows us to automatically refresh server data upon success.

- A standalone `createItem` function performs the actual Http Post request to the API server: it specifies the request body as JSON, serialises the new item into the request body, and throws an error if the response status is not OK (i.e., outside the 200-299 range).
- Inside `ItemCreatePage`, `useMutation` is configured with `createItem` as its `mutationFn`. Its `onSuccess` callback runs after a successful creation: it calls `queryClient.invalidateQueries` on the `'items'` query key so that `ItemListPage`'s data is refetched and shows the newly created item, then navigates to the `/items` route.

- The component passes `mutation.mutateAsync` as the `onItemChanged` prop to `ItemForm`, along with `mutation.error` and `mutation.isPending` as the `error` and `submitting` props. This is what drives the loading and error display inside the form.
- Any error thrown inside `createItem` (e.g. a failed network response) is automatically captured by `useMutation` and made available as `mutation.error`, so there's no need for a manual `catch` block here. React Query handles that for us.

App.tsx

Remember to add a Route entry in the App.tsx that points to the ItemCreatePage:



ItemAPIController create method

In addition, we need to add a new method in the api/Controllers/ItemController.cs:

```
43      [HttpPost("create")]
        0 references
44      public async Task<IActionResult> Create([FromBody] ItemDto itemDto)
45      {
46          if (itemDto == null)
47          {
48              return BadRequest("Item cannot be null");
49          }
50          var newItem = new Item
51          {
52              Name = itemDto.Name,
53              Price = itemDto.Price,
54              Description = itemDto.Description,
55              ImageUrl = itemDto.ImageUrl
56          };
57          bool returnOk = await _itemRepository.Create(newItem);
58          if (returnOk)
59              return CreatedAtAction(nameof(ItemList), new { id = newItem.ItemId }, newItem);
60
61          _logger.LogWarning("[ItemAPIController] Item creation failed {@item}", newItem);
62          return StatusCode(500, "Internal server error");
63      }
```

Line43: This attribute indicates that the method will handle **HTTP POST** requests sent to the create endpoint. The endpoint URL would be **API_URL/api/itemapi/create**, where **itemapi** is the name of the controller.

Line44: **public** means it is a public method (accessible from outside the class, making it a public API endpoint); **async** means the method is asynchronous, allowing for non-blocking operations (e.g., waiting for database operations).

Task<IActionResult>: The method returns a Task that represents an asynchronous operation returning an **IActionResult**. **IActionResult** is a common return type for ASP.NET Core action methods, allowing for various types of responses.

[FromBody] ItemDto itemDto: Indicates that the **itemDto** parameter should be bound from the body of the HTTP request. **ItemDto** is a Data Transfer Object used to transfer data between the client and the server.

Line36-Line49: if the **itemDto** object is null, it returns a **400 Bad Request** response

Line50-Line56: Converts the **ItemDto** into an Item entity. This involves mapping the properties from the DTO to the Item entity that will be stored in the database.

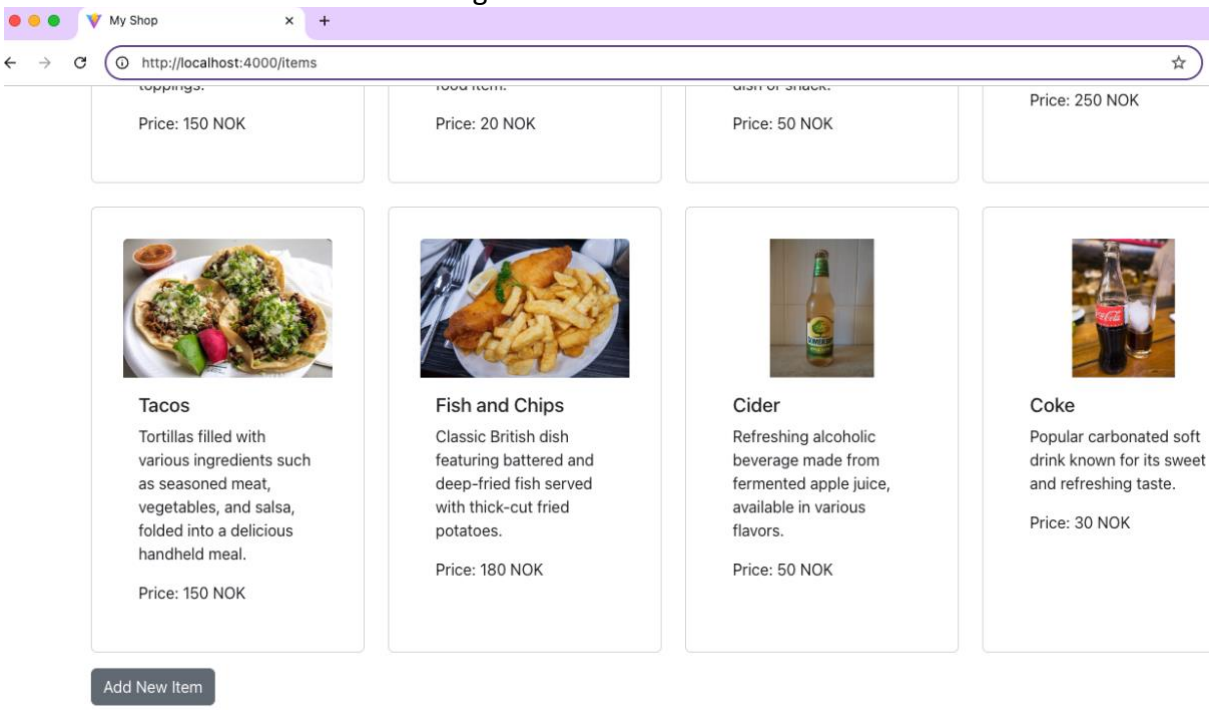
Line57: Calls a repository method Create to save the **newItem** to the database. The await keyword ensures that the method waits for the repository operation to complete before continuing.

Line58-Line59: If the creation operation is successful (**returnOk** is true), the method returns a **201 Created** response. The **CreatedAtAction** method generates a response with some context pointing to the newly created item's resource. The **nameof(ItemList)** indicates the action method used to retrieve the item details, and **new { id = newItem.ItemId }** provides route parameters for the location URL (What it means is not essential for now).

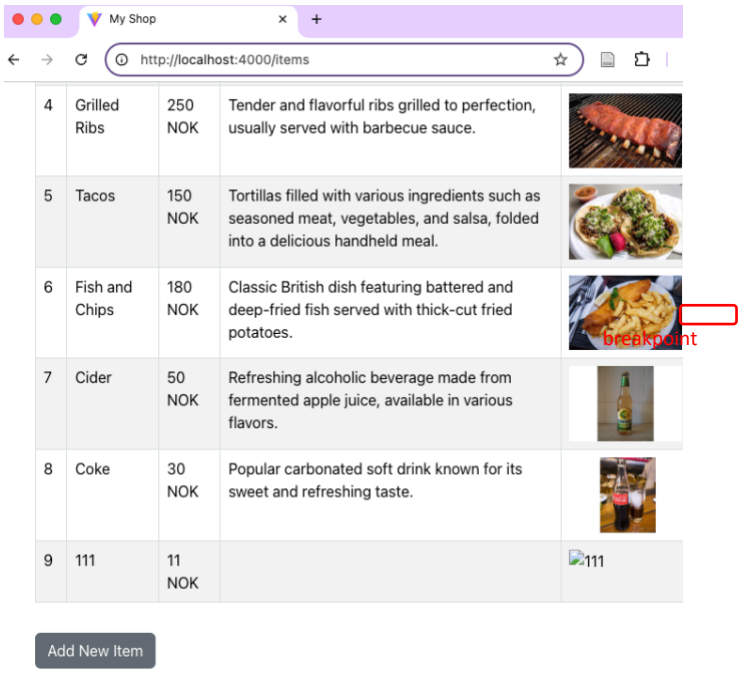
Line61-Line62: **Error handing**, If the creation operation fails (**returnOk** is false), it logs a warning with the details of the **newItem** and returns a **500 Internal Server Error** response with a generic error message.

Inspect the error logging and input validation

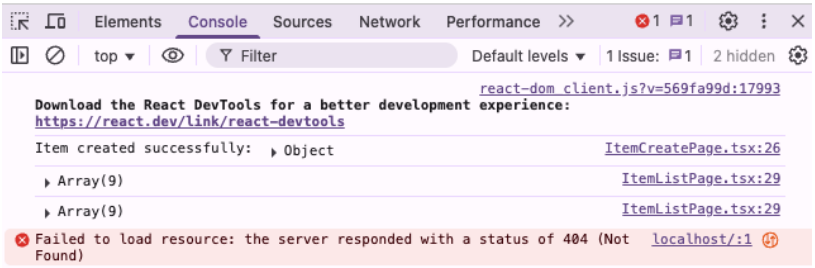
Run the project, **cd myshop**, **npm install** if needed, **npm run dev**. We can see that the new button is available on the ItemListPage.



Click the Add New Item and create a new item without image We can see that the new item is created.



You may see the error message in the Console like this in the **ItemListPage**.



The reason is that in **ItemTable.tsx** or **ItemGrid.tsx,** the img URL is set to **apiUrl** + **item.imageUrl**. The new item has **imageUrl** as “”, so its complete Url is namely **apiUrl**, which cannot retrieve any images.

We change these two files as such to display "No Image" when the image is not found.

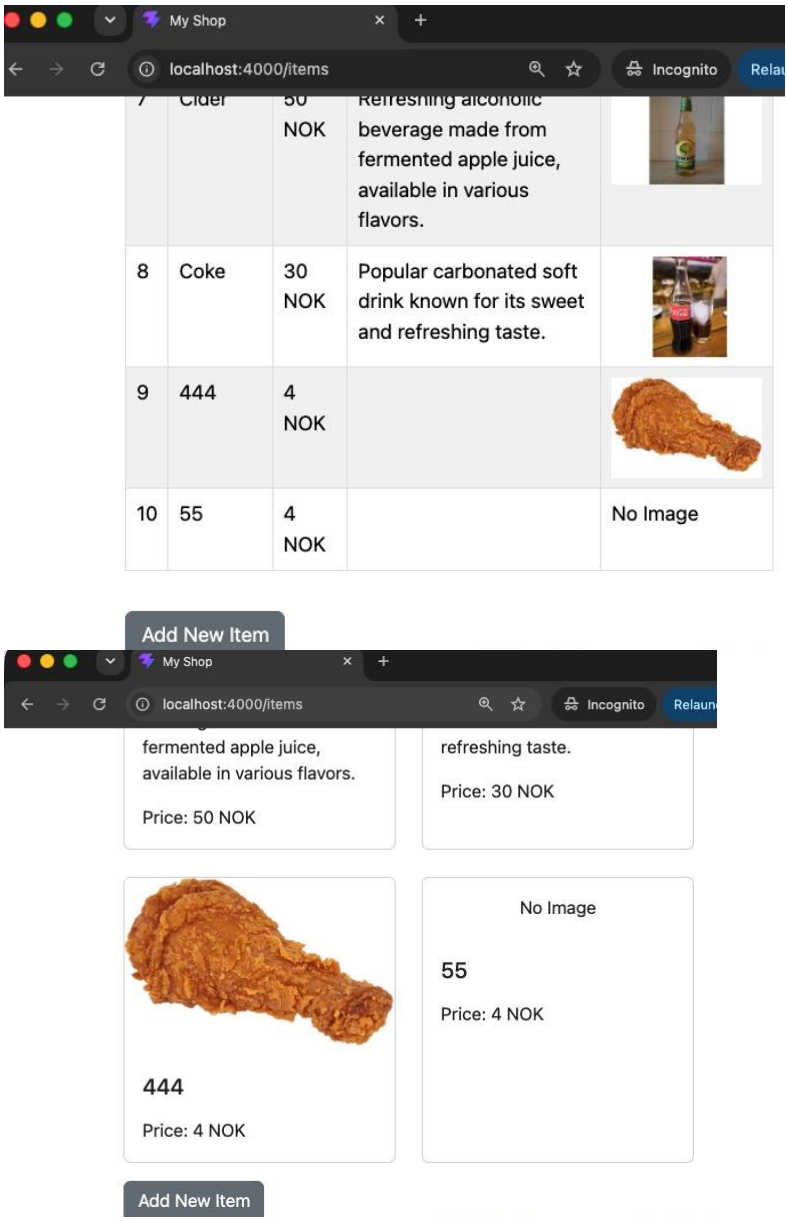
ItemTable.tsx Only the table body part is shown.

```
35      <tbody>
36        {items.map(item => (
37          <tr key={item.itemId}>
38            <td>{item.itemId}</td>
39            <td>{item.name}</td>
40            <td>{item.price} NOK</td>
41            {showDescriptions && <td>{item.description}</td>}
42            {showImages && (
43              <td>{item.imageUrl ? (<img
44                src={`${apiUrl}${item.imageUrl}`}
45                alt={item.name}
46                width="120"
47                onError={(e) => (e.currentTarget.style.display = 'none')}
48              /> ) : (<span>No Image</span>
49              )}</td>)}
50          </tr>
51        ))}
52      </tbody>
```

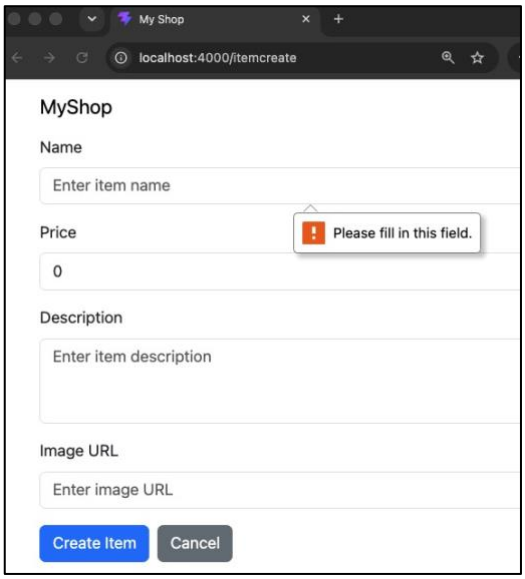
ItemGrid.tsx Only the return part is shown

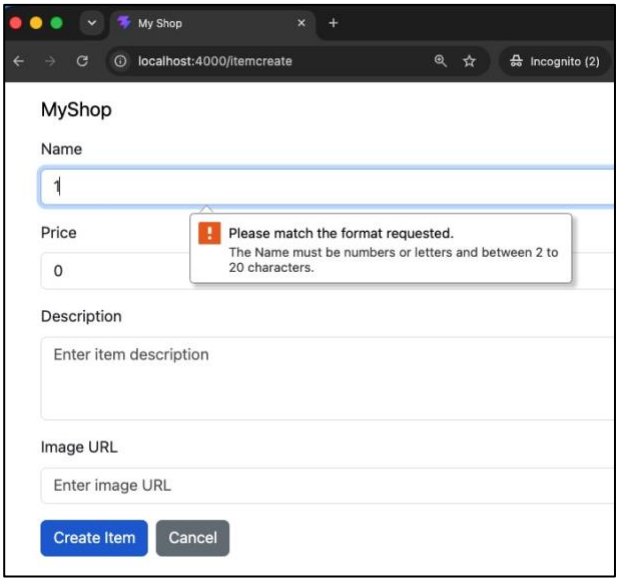
```
11  return (
12    <div>
13      <Row xs={1} sm={2} md={3} lg={4} className="g-4">
14        {items.map(item => (
15          <Col key={item.itemId}>
16            <Card className="h-100">
17              {item.imageUrl ? (
18                <Card.Img
19                  variant="top"
20                  src={`${apiUrl}${item.imageUrl}`}
21                  alt={item.name}
22                  onError={(e) => (e.currentTarget.style.display = 'none')}
23                />
24              ) : (
25                <div style={{ textAlign: 'center', padding: '1rem' }}>No Image</div>
26              )}
27              <Card.Body>
28                <Card.Title>{item.name}</Card.Title>
29                <Card.Text>
30                  {item.description}
31                </Card.Text>
32                <Card.Text>
33                  Price: {item.price} NOK
34                </Card.Text>
35              </Card.Body>
36            </Card>
37          </Col>
38        ))}
39      </Row>
40    </div>
41  );
```

Effect see:

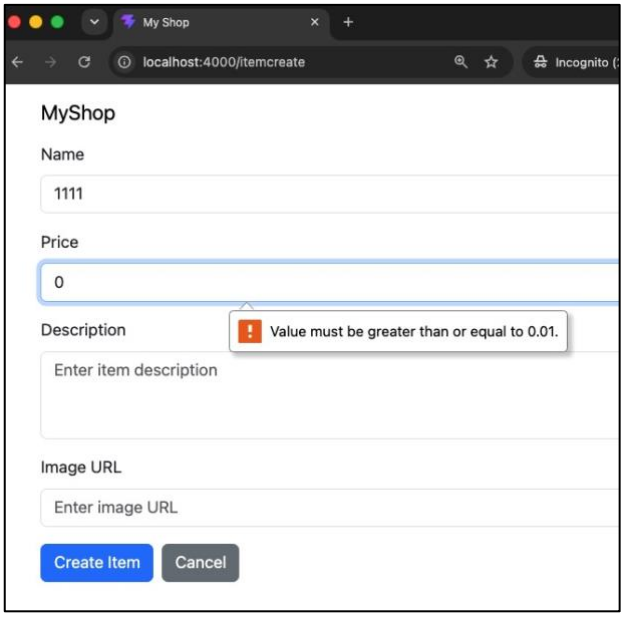

My Shop
localhost:4000/items
Incognito
7 Cider 50 NOK Refreshing alcoholic beverage made from fermented apple juice, available in various flavors.
8 Coke 30 NOK Popular carbonated soft drink known for its sweet and refreshing taste.
9 444 4 NOK
10 55 4 NOK No Image
Add New Item
My Shop
localhost:4000/items
Incognito
fermented apple juice, available in various flavors.
Price: 50 NOK
refreshing taste.
Price: 30 NOK
444
Price: 4 NOK
No Image
55
Price: 4 NOK
Add New Item

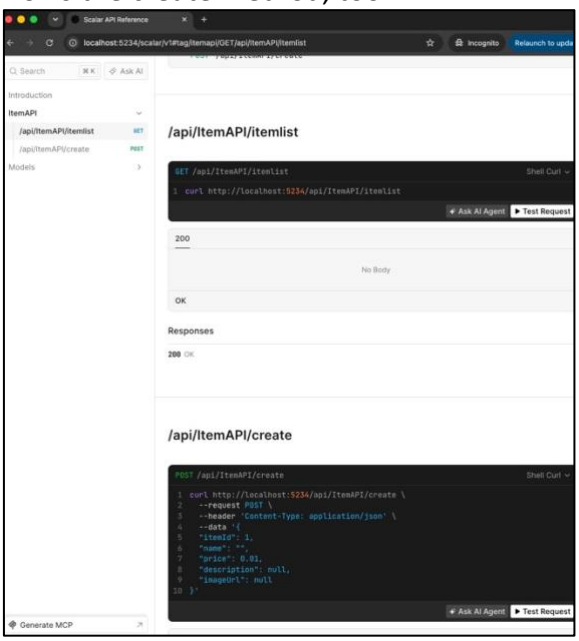
If you fill in something not following the input validation requirements in the form, the form will remind you of that:





Meanwhile, you can see that the Scalar UI now shows the create method, too.





Congratulations! Http Post and Item Creation is done! ☺